

Eclipse-10000

Qclipping

July 2026

1 Introduction

In this project, I have successfully recreated a full computer stack from scratch, starting with designing a 16-bit CPU in Logisim-Evolution using only logic gates and low-level components - such as splitters, bit extenders, pins, RAM, and ROM - with a custom ISA. Afterward, I created a separate 32-bit CPU using the SystemVerilog hardware description language, also featuring a custom ISA. Following that, I built my own compiler that translates my custom programming language into assembly, which then assembles into machine code instructions for that 32-bit CPU. My next step is developing an operating system for this CPU in my language.

2 Eclipse-10000

The whole project is named after this CPU I initially designed in Logisim, so let us start from it. It is a 16-bit multi-cycle CPU with a custom ISA (ISA16.txt file); I have built a simple 2-pass assembler for it.

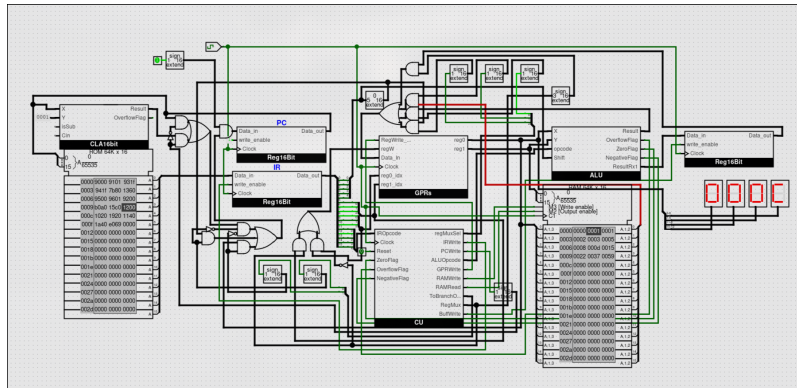


Figure 1: Diagram of the Core module calculating Fibonacci numbers

2.1 ALU

The ALU (Arithmetic Logic Unit) is one of the main CPU parts, so I gave it a lot of attention. It includes: AddSub module, Bitwise: AND, XOR, OR, NOT, SHL, SHR, and multiplication modules. It has a 4-to-16 bit decoder at its core. It also calculates all necessary flags: negative, overflow, and zero flags.

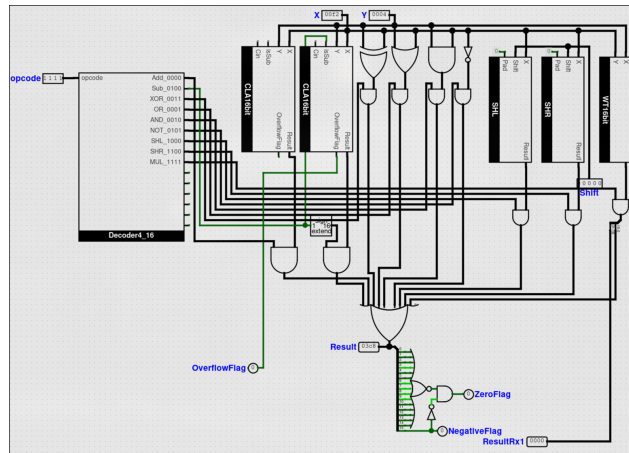


Figure 2: Diagram of the ALU module performing $0x00f2 * 0x0004$

2.1.1 16-bit CLA

The CLA (Carry Lookahead Adder) is an adder/subtractor pair. I split them into 4 smaller 4-bit CLAs, each one emitting Propagation and Generation signals, and then wired them up in the main module as I would wire a basic 4-bit CLA. That reduced the complexity of the building process while maintaining speed.

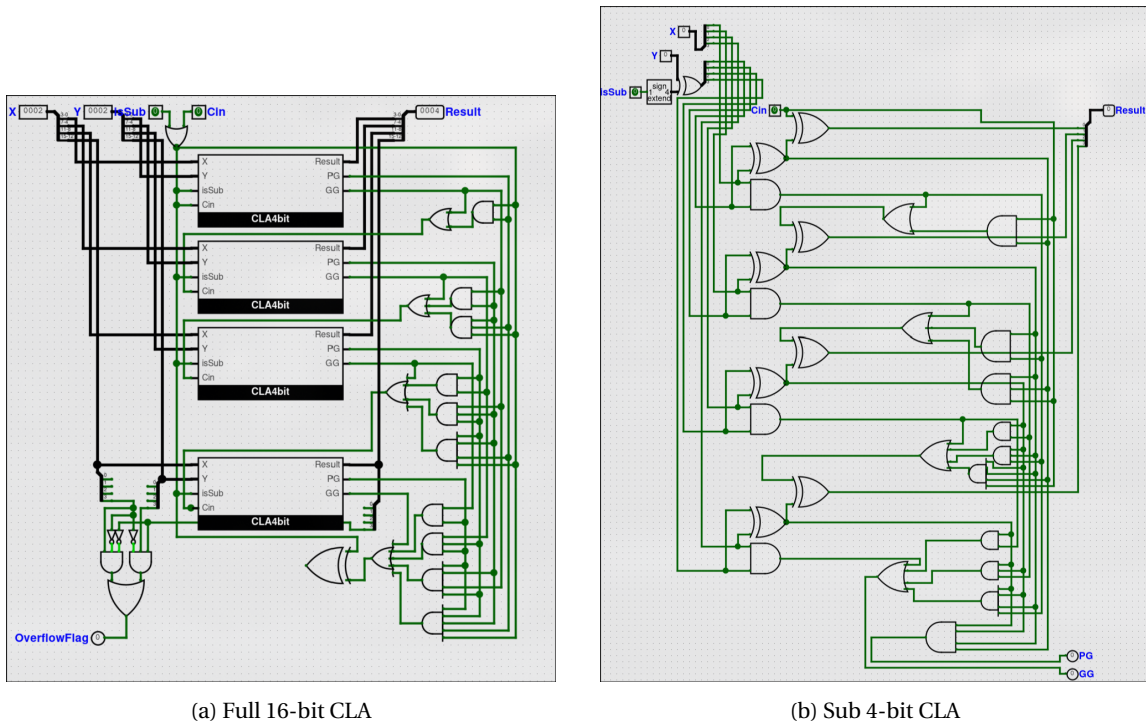


Figure 3: 4-bit CLA next to 16-bit CLA performing $2+2$

2.1.2 16-bit Barrel Shifter and Bitwise Operations

Standard bitwise operations (XOR, OR, AND, NOT) are performed using multi-bit input/output logic gates, whereas SHL and SHR are implemented using a barrel shifter in order to keep the shifting within 1 CPU cycle.

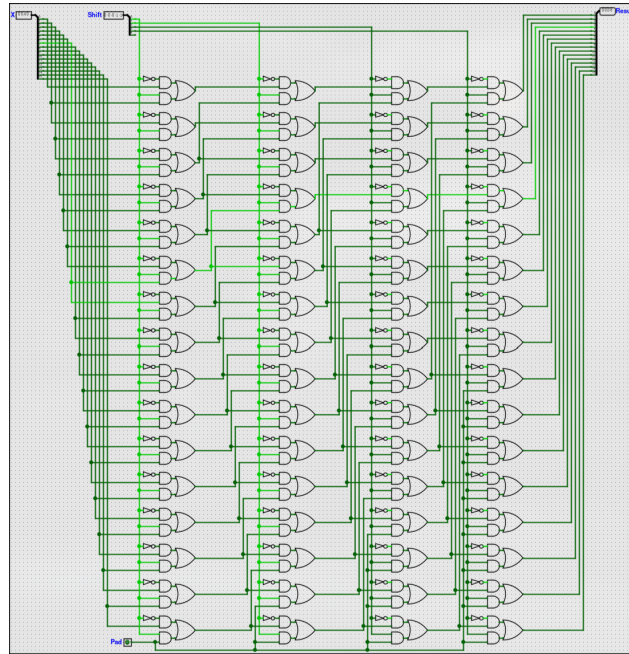
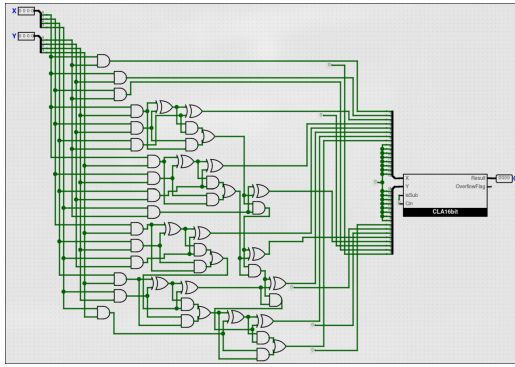


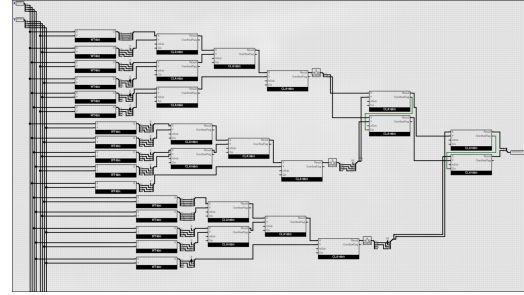
Figure 4: Diagram of the 16-bit SHR, shifting 0x0040 right by 0b0011

2.1.3 16-bit Wallace Tree

The Wallace tree is the multiplier module of the ALU. It is a better version of a regular array multiplier due to its usage of full and half adders as 3:2 and 2:2 compressors, and its time complexity is $\mathcal{O}(\log n)$. I have used the same principle as in the CLA, separating it into 4-bit sub-modules and combining them. However, it was more challenging to implement; I had to shift result bits and pad them with zeros in order to make it work. Multiplication of two 16-bit variables can result in a value up to 32 bits long; I solved this by storing the lower 16 bits in the first operand and the higher 16 bits in the second operand.



(a) Small 4-bit Wallace Tree

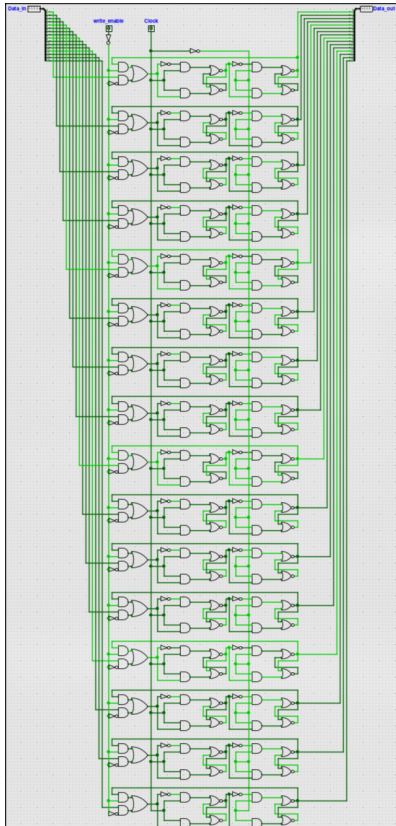


(b) Full 16-bit Wallace tree looks pretty evil

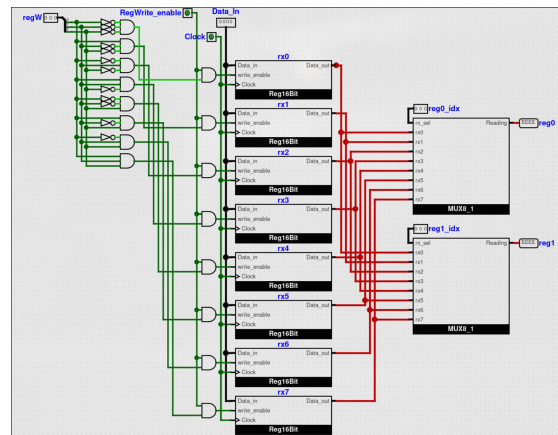
Figure 5: 4-bit Wallace Tree next to the full 16-bit one

2.2 GPRs

GPRs (General-Purpose Registers) are implemented using master-slave D flip-flops utilizing AND and NOR gates, totaling 8 GPRs. One register can be written to, and two registers can be read; reading is implemented using an 8-to-1 MUX.



(a) 1 16-bit register storing 0x1111



(b) Full 8-GPR Register File

Figure 6: All GPRs

2.3 CU

The CU (Control Unit) is a finite state machine operating in 6 distinct states. A multi-cycle design was chosen over a single-cycle one because it reduces the critical path length, vastly increasing the maximum possible clock speed.

- **State 0 (Fetch):** Loads an instruction into the Instruction Register (IR) and increments the Program Counter (PC) register.
- **State 1 (Decode):** Decodes the instruction and acts as a safety state.
- **State 2-5 (Execute-Writeback-Store-Read):** Based on the opcode, calculates particular control signals such as PCWrite for branching, regMux, RAM Read/Write, etc. Resets to State 0 after the instruction has finished, resetting at a different state depending on the instruction type.

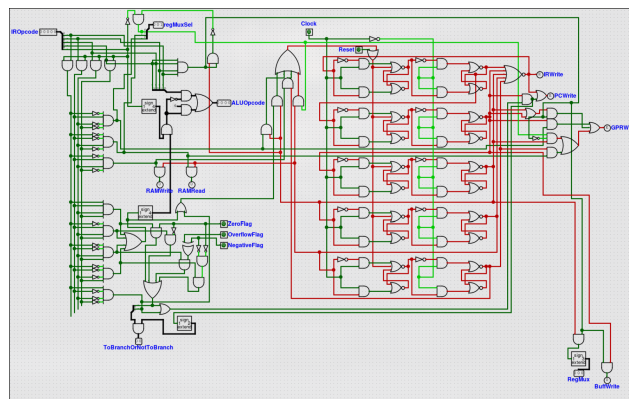


Figure 7: Diagram of the CU

3 Eclipse-100000

One more 0 indicates 32 bits. This is an actual CPU written in SystemVerilog. This CPU is multi-cycle as well.

3.1 Overview

Eclipse-100000 is a CPU with a completely custom architecture and ISA. The design is lightweight and optimized, focusing on native CPU operating-system support. Its main features are:

- 32 32-bit GPRs ($R_X0 - R_X31$) as well as a set of Special-Purpose Registers (SPRs)
- Dual-Privileged execution mode implemented using banking of some SPRs
- Memory Protection Unit with real-time memory access checks; invalid memory access leads to an immediate jump to the dedicated `MemoryFault` vector (0x70)
- Memory Topology (MMIO) for main memory, VRAM, and I/O
- Hardware Interrupt execution every set number of clock cycles that jumps to 0x64 for multitasking, alongside an integrated syscall vector at 0x64

3.2 ISA

The complete specification is available in `ISA32.txt`; at the same time, its core features are briefly outlined below. Instructions are 32-bit, encoded using various instruction types. The first 6 bits are always the opcode; subsequent bits depend on the exact type. I refer to the first register in the instruction as `rx0`, the second as `rx1`, and the third as `rx2`. `Imm` stands for immediate, and the number after it is the bit size specified (e.g., `imm10` is a 10-bit immediate).

Table 1: Eclipse-100000 Instruction Formats

Format	Bit Layout	Description / Use Case
R-Type	Opcode[31:26] rx0[25:18] rx1[17:10] Imm10[9:0]	Standard 2-operand ALU logic/shifts (10-bit unsigned imm)
B-Type	Opcode[31:26] rx0[25:18] rx1[17:10] rx3[9:2] imm2[1:0]	3-operand ALU operations with 2-bit loop modifier
I-Type	Opcode[31:26] rx0[25:18] rx1[17:10] Imm10[9:0]	Memory load/store (LDR/STR) with 10-bit signed offset
S-Type	Opcode[31:26] rx0[25:18] SPR_Sel[17:16] Imm16[15:0]	SPR (SP, LR, GP) ops with 16-bit signed immediate
L-Type	Opcode[31:26] rx0[25:18] Imm18[17:0]	Direct load (LOAD) & stack (PUSH/POP) (18-bit signed imm)
J-Type	Opcode[31:26] Imm26[25:0]	Jumps (JMP, CALL) & massive load (LMA) (26-bit imm)

Notes:

- **B-Type:** 2-bit loop modifier made specifically for `for` loops; it can be -1, +1, or +2 because I hard-coded `0b10` to be +2 instead of -2, specifically because I consider a +2 loop increment more useful than -2.
- **S-Type:** Currently 3 SPRs are accessible by the program: Stack Pointer (SP), Link Register (LR) for function return addresses, and General Pointer (GP) used to store initial SP location for globals (more on that in the FLARE-10K section).

3.3 Register Fragmentation

A central feature of the architecture is its hierarchical register fragmentation model, which directly simplifies code generation and type casting.

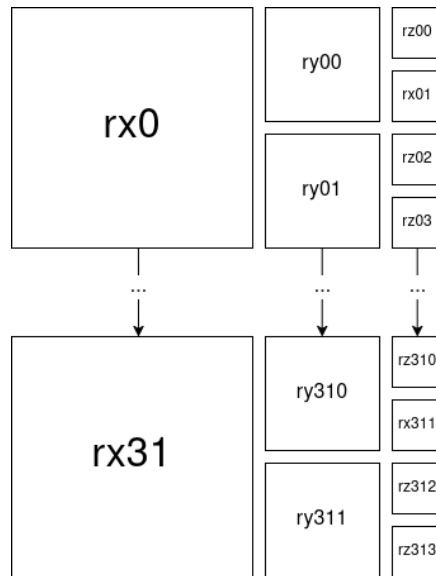


Figure 8: Representation of the principle of Fragmented Registers

Each 32-bit register (R_X) is fragmented into two 16-bit sub-registers (R_Y), each of which is further fragmented into two 8-bit (R_Z) sub-registers. The last digit in the name represents the sub-register specified, and all preceding digits represent the base register. For example, R_X16 is fragmented into $R_Y16[0-1]$ and $R_Z16[0-3]$; thus, the highest 8 bits of R_X16 would be R_Z163 . Bear in mind, though, that they are not separate registers but rather parts of one full register, so modifying a sub-register directly modifies its parent register.

3.4 Control Unit

Eclipse-100000 has a total of 10 Finite State Machine states:

```
typedef enum logic [3:0] {
    FETCH,
    DECODE,
    ALU_EXE,
    BRANCH,
    LOAD,
    READ_DATA,
    EXCEPTION,
    TIMER_INTERRUPT,
    MEM_FAULT,
    STORE
} fsm_states;
```

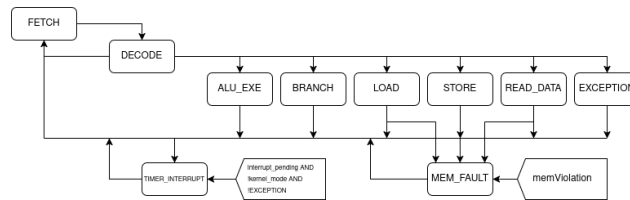


Figure 9: Flowchart of the CPU's FSM; squared arrows represent conditions

- **FETCH:** Fetches the instruction and increments PC by 4 because memory uses byte addressing, so each instruction is 4 bytes.
- **DECODE:** Deduces the next state based on opcode and calculates the effective address ($PC + \text{RegX}$) since the ALU is idle. Control flow instructions (CALL, RET, JR, JMP) execute during the decode phase to save clock cycles and increase performance.
- **ALU-EXE:** The ALU performs calculations and writes the result back.
- **BRANCH:** Computes whether the branch condition is met or not; if yes, it branches.
- **LOAD:** Loads from memory based on whether it is an SPR read or a regular memory read.
- **READ-DATA:** Reads data from memory; also used for POP.
- **EXCEPTION:** Raises privileges to kernel mode and saves current PC to the respective vector.
- **TIMER-INTERRUPT:** Interrupt that happens every set amount of cycles; raises privileges to kernel mode and saves current PC to the respective vector.
- **MEM-FAULT:** Immediately disables memory access, raises privileges, saves PC, and jumps to the respective vector.

- **STORE:** Stores data in memory; also used for PUSH.

4 FLARE-10K

FLARE-10K is a statically and strongly typed, AOT-compiled imperative systems language. It is designed to write an OS on; it is generally a C-like language, but considerably different. The documentation is in the `docs/flare.md` file; therefore, this chapter will focus on explaining implementation specifics. I encourage readers to review the documentation (`docs/flare.md`) first. The following sections will outline and explain each part of the compiler.

4.1 Lexical Analysis

The lexer is the first part of the compiler pipeline. It works by looking at the current character and occasionally peeking at the next one to determine the type of Token it is looking at.

There is one unique feature of the Lexer: inline Lexing. Inlines are lexed just as they are; however, due to the rather large number of Tokens an inline statement requires, all necessary tokens are pushed to a queue in their respective order, which then get pushed to the final vector of tokens.

4.2 Parser

The parser is Recursive Descent LL(1) with localized LL(2) fragments. The parser generates a full Abstract Syntax Tree (AST). The precedence table for my language's grammar is presented below; I took the idea from the book *Crafting Interpreters* by Robert Nystrom. Each block represents a grammar construct; the height represents precedence, meaning higher blocks at some point descend to lower blocks. It is LL(2) for struct/regarch declarations because they consist of an identifier followed by another identifier, though this could have been implemented without using LL(2).

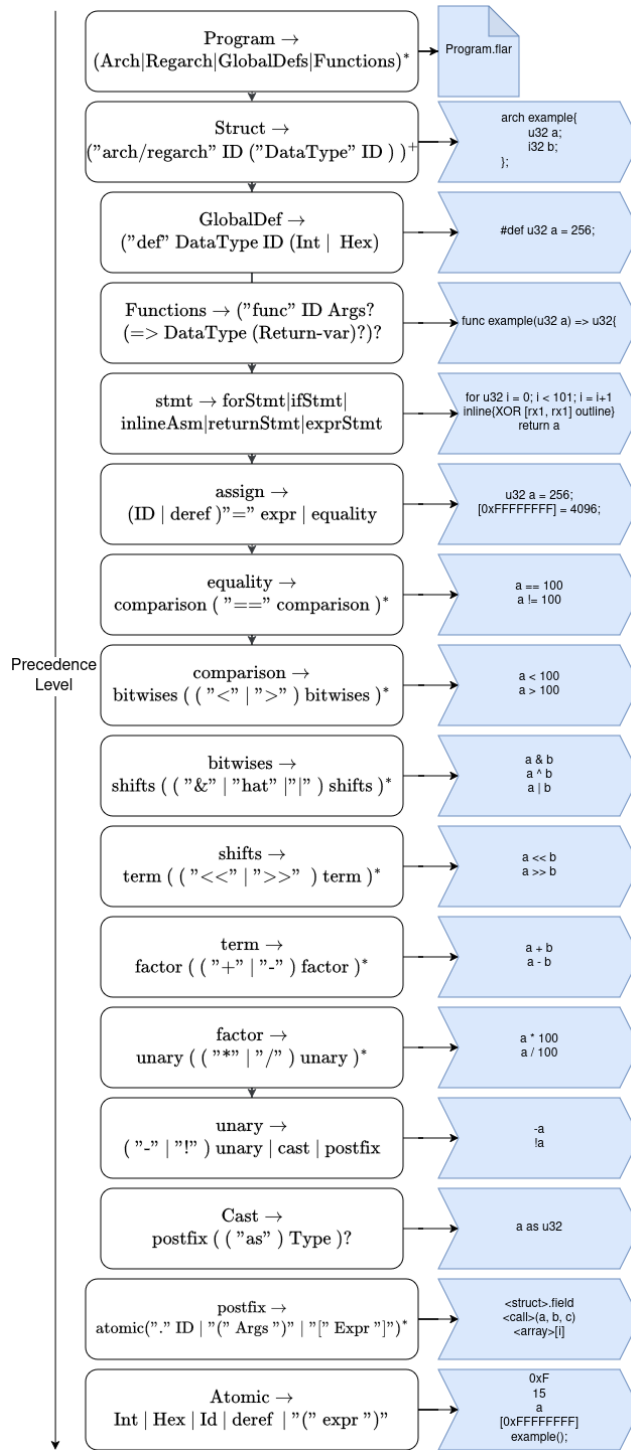


Figure 10: Precedence Table of FLARE-10K

I believe the precedence table is comprehensive enough to explain the main pipeline of the parser.

4.3 Semantic Analysis

The semantic analyzer is rather minimal; however, it still performs basic functions. It iterates through the AST generated by the parser and inserts variables, functions, structs, globals, scopes, loop depths, and the current return name into the general Semantic Struct. It then looks for uninitialized variables, unidentified structs, wrong parameter counts, and other issues. It might misbehave occasionally, but generally, it is a good tool to ensure the program is valid.

4.4 Intermediate Representation (IR)

As I mentioned in the documentation, the IR was specifically made fairly general for two main purposes:

1. A possible backend for other architectures, and
2. Preparing for later optimization.

The IR lowers (I prefer the term reduces, or even better, flattens, but I will stick to the official term) the Abstract Syntax Tree into Three-Address Code (TAC).

4.4.1 IROperand

An IROperand is literally an operand of an IRInst. There are several IROperand types:

- Signed/unsigned constants – Just scalar values like 10.
- Var – User-declared variable.
- Temp – Value generated by the compiler.

IROperands do not get registers assigned at this point; the IR just specifies that they are registers by either using a variable name or allocating one of infinitely many virtual registers to them (which is often done for constants). Codegen then allocates actual hardware registers to them. This is similar to other IROperands: they do not actually get memory offsets; the IR just determines in what kind of memory they live, and codegen then handles offsets and everything else.

4.4.2 lvalue/rvalue Duality

Depending on the expression type, it gets parsed differently. In one scenario, it gets parsed using a reduce-expression function that produces a value; in another case, by a lower-lvalue function that produces an address. It is used for getting the address of constructs such as structs, arrays, dereferences, and others.

However, it is worth mentioning that since regarches literally do not have an address, they are parsed as RefFieldRead and RegFieldWrite, which determine what part of the regarch we are accessing and, based on that, output the sub-register we are dealing with (fragmented registers come into play again).

4.4.3 Functions

Actually, the IR does work with actual registers in the calling stage: it puts arguments in the first 4 registers, meaning there might be up to 16 arguments passed via these 4 registers assuming they are all 8 bits, like u8 or bool. Regarches are placed into one of the 4 input registers; if the first 4 registers are already filled, though, regarches get spilled onto the stack like regular arches or any other variable. As for regular structs and arrays, their addresses get pushed onto the stack, defaulting to pass-by-reference.

Now, return variables are actually pretty simple since they are similar to arguments; they are implemented in a similar way, but with NULL initialization and optional pinning. Also, I used the principle of a void-return function (which by default returns nothing) and applied it to functions with return variables.

4.4.4 Notes

Regarding `AntiEqual`, `Equal`, etc. naming: for example, `Less` becomes `AntiLess`, which in codegen becomes "greater than."

4.5 Code Generation

Now, code generation is the most complicated part of FLARE-10K, so I am going to outline its main features at a high level.

4.5.1 Register Allocation

To map an unlimited number of virtual registers generated by the IR onto real hardware registers, I use the Chaitin-Briggs optimistic graph-coloring algorithm.

1. Liveness analysis & interference graph The first step is analyzing a variable's liveness. A variable is live at any point in the program if it is going to be read before it gets overwritten (killed) or the program ends. I scan each basic block (the largest sequence of code without branching) from bottom to top - because variables are typically read on the right-hand side of an expression - and determine which variables are live simultaneously.

Afterward, we use the collected data to construct an interference graph where nodes represent a variable's live range, and an edge between nodes indicates that variables are live at the same time.

2. Node removal and spilling The next step is to repeatedly remove any node with a degree less than K , where K , in my architecture, is the register capacity in bytes (120 bytes). Because such nodes have fewer neighbors than available register bytes, they are mathematically and logically guaranteed to get a register. Fortunately, removing any node results in the degree of all neighbors being decreased by 1, thereby simplifying the process. Removed nodes get pushed onto the stack.

However, the algorithm might stall, meaning there are no nodes with a degree $< K$. In that case, a spill weight for each node is computed based on its usage frequency and presence in a loop (if a variable is read in a loop, $10^{\text{loop depth}}$ is added to its spill weight). Then, the node with the lowest spill weight gets optimistically pushed because it is less critical and will decrease the degree of all neighboring nodes, thereby increasing the chances of more important variables being allocated.

3. Coloring Finally, variables are popped off the allocation stack one by one to assign physical fragmented registers to them. 16-bit variables are aligned to fit full R_Y registers (in either the lower or higher 16 bits of a register). In most cases, a variable will get allocated - even if it was optimistically pushed - because neighbors often share the same register. However, in rare cases where that does not happen, the last resort comes into play and variables get spilled to the stack.

4. Spill Rewriting and Re-allocation Spilled variables rewrite the IR: every time they are read, we load them into a register, and every time they are written, they are stored to memory. Therefore, this changes liveness and it has to be recomputed. This loop happens up to 100 times as a safety valve.

There is much more to this algorithm. If you are interested, I encourage you to explore it further or read my implementation in the `codegen.rs` file, where I tried commenting it as comprehensively as possible.

I also want to mention that pinned variables and regarches skip the allocation process, and their assigned registers are marked as used. Also, registers get allocated from the top down (from `rx29` down) to minimize interference with argument passing.

4.5.2 Temporary Registers

Unfortunately, I had to reserve rx31 and rx30 as temporary registers for code generation because we need a place to store hardcoded constants. Immediately after usage, those registers are XORed with themselves so they become 0, unless the next instruction overwrites them (LOAD/LMA) - a small optimization that really matters in array initialization.

4.5.3 Casting

Casting is very convenient - it heavily utilizes register fragmentation, so a simple MOV instruction is enough for most cases. Moving to larger registers automatically zero-extends/upcasts, while moving to smaller registers automatically truncates to the lower bits. There is one special case: signed upcasting requires an initial SHL followed by an SRA.

4.5.4 Globals

Global variables or structs are stored at a positive SP offset. This works by first calculating the total size of all globals, decreasing the stack pointer by that amount, and then setting the General Pointer SPR (GP) to the current SP. Thus, globals live just before the main stack of the program at a positive GP offset. Global arrays are also supported; they work like regular global variables, loading elements sequentially at positive GP offsets.

4.5.5 Branching

Due to the lack of "jump if less than or equal" (or its counterpart), $\leq$ and $\geq$ operations compile down to two instructions: a branch if less/greater, followed by a branch if equal. Fortunately, branching takes only 2 CPU cycles on my architecture, making this a negligible factor in execution speed while saving opcode space for other ISA instructions.

4.5.6 Non-Leaf Functions

A function is non-leaf if it calls another function. This distinction is important because there is only one Link Register (LR), which can store only one return address. Therefore, LR must be saved before calling another function within a non-leaf function; this is resolved by simply pushing it onto the stack right before the call and popping it right after.

4.5.7 Call Saves

During a function call, not every variable or register gets spilled to the stack - only live ones do. This is implemented by walking through the basic block containing the call in reverse and using the same mechanism used for liveness analysis to compute which values need to be saved and subsequently restored.

4.5.8 Stack Argument Resolution

Arguments that don't fit into the first four argument registers are pushed immediately before CALL by the caller function. On the callee's function side, these are not addressed relative to the incoming stack pointer directly; instead, each is represented in the IR as an `IncomingArgSlot(idx)`, which codegen lowers to `SP + local_frame_size + idx * 4` once the callee's own prologue has finished decreasing stack for its own spills. This offset has to be computed after `local_frame_size` is known, since every byte the callee reserves for its own frame pushes the caller's pushed arguments further away from SP.

4.6 Trade-offs

There are currently several limitations of the compiler that I am planning to fix:

1. No multi-expression `if` statements (`&&` and `||` are not implemented yet). This is easily worked around by nesting loops and utilizing `else if` statements.
2. no callee-saved registers exist at all - everything is caller-saved, though I would hardly call it a trade-off, its just a design decision
3. Global array's elements must be known literals at compile time
4. No compile time global evaluation, for example `#def u32 a = 5+10;` is not valid.
5. Currently, there are no dedicated optimization passes. (Though register allocation itself yields significant speedups.)
6. FLARE-10K lacks syntactic sugar such as `+=` and `-=`.
7. Functions cannot return arrays, nor can they accept structs or arrays by value (non-reference).
8. No hardware division or modulo instructions, as I refuse to add them to the ALU without implementing them using purely primitive logic gates in Logisim first.
9. The allocator halts after 100 rewrite passes, which usually indicates an allocation bug.